Problem 9: Final challenge: chunked KDA

GStar Bootcamp

Abstract

Build chunked KDA forward execution using your earlier student kernels.

Starter file: `problem_9.py`. **Checker:** `python autograder.py -p9`. **Reading path:** Overview, P4, P6, P8. Next: Verification.

Mathematical background: compose an exact state transformation

Each KDA token applies an affine map $\mathcal{F}_t(\mathbf{S}) = \mathbf{T}_t\mathbf{S} + \mathbf{B}_t$, with

$$\mathbf{T}_t = (\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top)\mathbf{G}_t, \qquad \mathbf{B}_t = \beta_t \boldsymbol{k}_t \boldsymbol{v}_t^\top.$$

For two consecutive maps, composition gives

$$(\mathbf{T}_2, \mathbf{B}_2) \circ (\mathbf{T}_1, \mathbf{B}_1) = (\mathbf{T}_2\mathbf{T}_1,\ \mathbf{T}_2\mathbf{B}_1 + \mathbf{B}_2).$$

Composition is associative but generally not commutative. In principle, tokens can be grouped into chunks without changing their order. Forming arbitrary dense transition products would be expensive; the KDA representation exploits diagonal transport and low-rank writes instead [1].

The resulting chunk map has the compact form

$$\mathbf{S}_{\text{out}} = \underbrace{\left[\operatorname{Diag}(e^{\boldsymbol{b}_C}) - (\mathbf{K}^{\text{end}})^\top \mathbf{W}\right]}_{\text{entry-memory transition}} \mathbf{S}_{\text{in}} + \underbrace{(\mathbf{K}^{\text{end}})^\top \mathbf{U}}_{\text{new-information term}}.$$

This expression is equivalent to first computing actual corrections $\mathbf{P} = \mathbf{U} - \mathbf{W}\mathbf{S}_{\text{in}}$ and then transporting them to the exit. It explains why WY factors depend only on local inputs while pseudo-values depend on the entry memory. Output reads require saved entry states as well as local corrected writes, so an exit state cannot replace an entry state.

P9 therefore assembles mathematical objects already introduced: P4's log-prefix transport, P6's triangular correction system and P8's channel-aware scores. The equations below specialize those objects to KDA. Passing the same final state into a second call must reproduce continuous execution, because a completed state is precisely the input to the next affine transformation.

Before coding. Identify which buffers can be built without an entry state, which operation must wait for it, and why the output needs inclusive scores even though the correction system is strict.

Derive the gated correction system

KDA combines Problem 6's correction system with Problem 8's channel-aware scores. Let $\boldsymbol{b}_r$ be the channel log-prefix. The decayed state just before the correction at row r is

$$\overline{\mathbf{S}}_r = \operatorname{Diag}(e^{\boldsymbol{b}_r})\mathbf{S}_{\text{in}} + \sum_{u<r} \operatorname{Diag}(e^{\boldsymbol{b}_r - \boldsymbol{b}_u})\boldsymbol{k}_u \boldsymbol{u}_u^\top. \tag{9.1}$$

Substituting this into the key prediction produces a scalar interaction per earlier correction:

$$\boldsymbol{k}_r^\top \operatorname{Diag}(e^{\boldsymbol{b}_r - \boldsymbol{b}_u})\boldsymbol{k}_u = F(\mathbf{K}, \mathbf{K})_{ru}.$$

Set $\mathbf{G}_{kk} = \mathrm{stricttril}(F(\mathbf{K}, \mathbf{K}))$ and $\mathbf{K}^{\mathrm{start}} = \mathbf{K} \odot e^{\boldsymbol{b}}$. The residual system is

$$(\mathbf{I} + \mathbf{D}_\beta \mathbf{G}_{kk})\mathbf{P} = \mathbf{D}_\beta(\mathbf{V} - \mathbf{K}^{\mathrm{start}}\mathbf{S}_{\mathrm{in}}). \tag{9.2}$$

Define the KDA compact factors [1]:

$$\mathbf{A} = (\mathbf{I} + \mathbf{D}_\beta \mathbf{G}_{kk})^{-1}, \tag{9.3}$$

$$\mathbf{W} = \mathbf{A}\mathbf{D}_\beta \mathbf{K}^{\mathrm{start}}, \qquad \mathbf{U} = \mathbf{A}\mathbf{D}_\beta \mathbf{V}, \tag{9.4}$$

$$\mathbf{P} = \mathbf{U} - \mathbf{W}\mathbf{S}_{\mathrm{in}}. \tag{9.5}$$

The beta multiplication acts on rows of $\mathbf{G}_{kk}$, not columns. The key–key score builder is unscaled.

Exit state and output

Let $K^{\mathrm{end}}_{ud} = K_{ud} e^{b_{Cd} - b_{ud}}$ and $\mathbf{Q}^{\mathrm{start}} = \mathbf{Q} \odot e^{\boldsymbol{b}}$. Then

$$\mathbf{S}_{\mathrm{out}} = \mathrm{Diag}(e^{\boldsymbol{b}_C})\mathbf{S}_{\mathrm{in}} + (\mathbf{K}^{\mathrm{end}})^\top \mathbf{P}, \tag{9.6}$$

$$\mathbf{O} = \left[\mathbf{Q}^{\mathrm{start}}\mathbf{S}_{\mathrm{in}} + \mathrm{tril}(F(\mathbf{Q}, \mathbf{K}))\mathbf{P}\right]. \tag{9.7}$$

Query scaling is omitted from these equations. In code, apply the optional `scale` consistently to both output contributions; never scale the WY system.

Problem 9: forward pipeline and buffer ownership

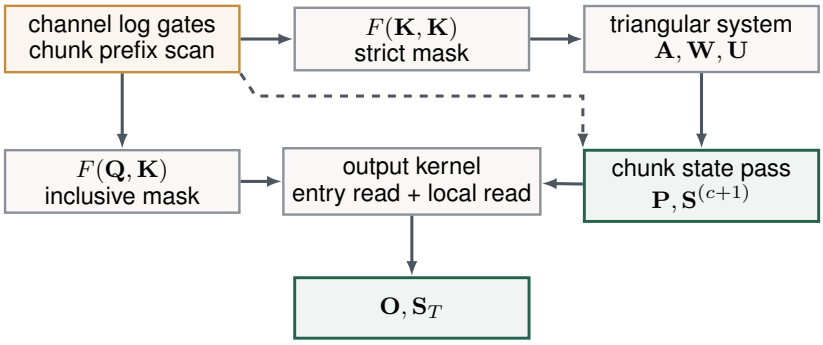


Figure 1. KDA forward dependencies. Score and WY work is chunk-local; pseudo-values depend on each chunk's entry state. The final output reads saved states and pseudo-values. Dashed input supplies prefixes to the state pass.

Buffer shapes

With $N = \widetilde{T}/C$, use the following logical layouts: A flattened score layout $(B, H, \widetilde{T}, C)$ is equivalent.

Table 1. Buffer shapes.

Buffer	Shape	Role
Log prefixes	$(B, H, \widetilde{T}, D_k)$	Chunk-relative decay
Key–key / output scores	(B, H, N, C, C)	Strict / inclusive interactions
$\mathbf{W}$	$(B, H, \widetilde{T}, D_k)$	Entry-state correction factor
$\mathbf{U}, \mathbf{P}$	$(B, H, \widetilde{T}, D_v)$	Base and actual pseudo-values
States	$(B, H, N+1, D_k, D_v)$	Every chunk's entry and exit state
Output	$(B, H, \widetilde{T}, D_v)$	Trimmed to real sequence length

Launchers provide the actual allocation and offsets.

Key-tiled pseudo-value reduction

If $D_k = 128$, $\mathbf{WS}_{\text{in}}$ spans two key tiles. Load $\mathbf{U}$ once, subtract both contributions, and only then finalize $\mathbf{P}$. Updating a state tile before the complete pseudo-values are known mixes entry and exit states and breaks Eq. (9.5).

Problem 9: implementation steps and final checks

1. **KDA WY representation (55 points).** Reuse Problem 8's strict key–key builder and Problem 6's triangular primitive. Complete Eqs. (9.3)–(9.4), transforming keys toward the chunk start for $\mathbf{W}$.
2. **Pseudo-values and state updates (55 points).** Complete the key-tiled reduction for $\mathbf{P}$. Transform keys toward the chunk end, decay the entry state by the final channel prefix, and write the exit state using Eq. (9.6).
3. **Output kernel (40 points).** Reuse the inclusive query–key builder. Multiply its score tile by $\mathbf{P}$, add the entry-state read with transformed queries, and store output with the correct scale and dtype.
4. **Dependencies and state passing (25 points).** Explain the launch dependencies and demonstrate segmented execution at an interior token that is not a chunk boundary.

```
# _chunk_wy_representation
# TODO P9.A: beta-weighted strict system and triangular inverse.
# TODO P9.B: entry-decayed keys in W; beta-weighted values in U.

# _chunk_state_forward
# TODO P9.C: subtract ALL W @ entry-state key-tile contributions.
# TODO P9.D: decay entry state; add exit-decayed key writes.

# _chunk_output_forward
# TODO P9.E: inclusive scores @ pseudo-values.
# TODO P9.F: entry-decayed query read; combine and scale once.
```

Reduction checks

- **All channel gates zero:** the operator becomes DeltaNet.
- **Same gate for every channel:** the operator becomes Gated DeltaNet.
- **All $\beta = 0$:** no corrections are written; only the initial state decays and is read.
- **Zero initial state and $\beta = 0$:** outputs and final state are zero.

Also test $D_k \neq D_v$, repeated keys, strong unequal gates, partial chunks, and nonzero initial state. Compare both outputs and final state with the independent KDA recurrence, rather than only comparing with your recurrent kernel.

Completion criterion

Your KDA pipeline implements the same recurrence in two execution forms. It uses stable gated scores, bounded chunk-local matrices, correctly timed states, and query scale exactly once. Every output and final-state check passes the documented tolerance, and every buffer read has been initialized.

Problem 9: a two-channel KDA example

Take $D_k = 2$, $D_v = 1$, $C = 2$, and

$$\mathbf{S}_{\text{in}} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}, \quad \boldsymbol{k}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \boldsymbol{k}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad (v_1, v_2) = (3, 2), \quad (\beta_1, \beta_2) = (1/2, 1).$$

Let the channel decays be $e^{\boldsymbol{g}_1} = (1/2, 1/4)^\top$ and $e^{\boldsymbol{g}_2} = (1/4, 1/2)^\top$.

Follow the recurrence first

At token 1, forgetting gives $(1,1)^\top$. The keyed prediction is 1, the correction is $u_1 = 1$, and memory becomes $(2,1)^\top$. At token 2, forgetting gives $(1/2,1/2)^\top$. The keyed prediction is $1/\sqrt{2}$, so $u_2 = 2 - 1/\sqrt{2}$ and

$$\mathbf{S}_2 = \begin{bmatrix}1/2\\1/2\end{bmatrix} + \frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}\left(2 - \frac{1}{\sqrt{2}}\right) = \begin{bmatrix}\sqrt{2}\\\sqrt{2}\end{bmatrix}.$$

For queries $\boldsymbol{q}_1 = (1,0)^\top$ and $\boldsymbol{q}_2 = (0,1)^\top$, the outputs are 2 and $\sqrt{2}$.

Check the chunk quantities

Prefix decays are $(1/2,1/4)^\top$ at row 1 and $(1/8,1/8)^\top$ at row 2. The strict key–key interaction is

$$F(\mathbf{K},\mathbf{K})_{21} = \frac{1}{4\sqrt{2}}.$$

Only the first channel contributes, because $\boldsymbol{k}_1$ has zero second coordinate. This is a useful test that the gate difference remains inside the channel reduction. The triangular inverse is

$$\mathbf{A} = \begin{bmatrix}1 & 0\\ -1/(4\sqrt{2}) & 1\end{bmatrix}.$$

Your actual pseudo-values must be $(1,\ 2 - 1/\sqrt{2})^\top$. The exit formula combines initial-memory decay $(1/4,1/2)^\top$, transported first correction $(1/4,0)^\top$, and the un-decayed current correction. It reproduces $\mathbf{S}_2$ above.

Use this example as a derivation check

Compute $\mathbf{W}$ and $\mathbf{U}$ from the published formulas, then verify $\mathbf{U} - \mathbf{W}\mathbf{S}_{\text{in}}$ produces the two corrections. Compare outputs and final state separately. These tiny mathematical dimensions are for hand calculations and the reference; the GPU starter supports 64/128 channels.

Problem 9: assembly order and the completed forward operator

Required dependency order

1. Pad input tokens neutrally, preserving the real length for output trimming.
2. Run P4's chunk-local channel scan to obtain FP32 log prefixes.
3. Run P8's strict key–key score builder with scale 1.
4. Use P6's inverse inside the KDA WY kernel to produce A/W/U.
5. Scan chunk entries; compute each chunk's complete pseudo-values before writing any exit-state tile.
6. Build inclusive query–key scores using P8 and the requested output scale.
7. Compute outputs from saved entry states and actual pseudo-values; trim output and return the final state when requested.

The inclusive scores may be built earlier because they depend only on local query/key/gate data. The output kernel must wait for both its scores and its saved entry state/pseudo-values. The supplied launcher follows a simple valid order rather than overlapping independent work.

Read the launcher using the same buffer names

The supplied wrapper separates these stages with blank lines. Its main intermediates follow this order; the output allocation, launch arguments, and trimming remain in `problem_9.py`.

```
gate_cumsum = chunk_cumsum(gate)
kernel_matrix = gated_scores(
    key, key, gate_cumsum, scale=1.0, strict=True,
)

w, u, inverse = _wy_representation(
    key, value, gate_cumsum, beta, kernel_matrix,
)
states, pseudo_values = _chunk_states(
    key, w, u, gate_cumsum, initial_state,
)

scores = gated_scores(
    query, key, gate_cumsum, scale=scale, strict=False,
)
```

Here `gate_cumsum` stores $\boldsymbol{b}$, `inverse` stores $\mathbf{A}$, and `pseudo_values` stores $\mathbf{P}$. The output kernel receives `states`, `scores` and `pseudo_values`; neither the WY base values `u` nor the strict `kernel_matrix` can replace the corresponding output input.

The key reduction is a read-before-write dependency

For $D_k = 128$, partition the key axis into $\mathcal{D}_0$ and $\mathcal{D}_1$ and compute

$$\mathbf{P} = \mathbf{U} - \mathbf{W}_{:,\mathcal{D}_0}(\mathbf{S}_{\text{in}})_{\mathcal{D}_0,:} - \mathbf{W}_{:,\mathcal{D}_1}(\mathbf{S}_{\text{in}})_{\mathcal{D}_1,:}.$$

Both terms must read the same entry memory. Only after this expression is complete may either state tile advance. This dependence explains the two distinct key loops in `_chunk_state_forward`: the first subtracts all predictions; the second transports keys and writes all exit tiles.

Scaling in the supplied KDA pipeline

The strict key–key buffer is unscaled. The inclusive query–key buffer includes `scale`. The output kernel's entry-query load also includes `scale`. Add the two contributions directly. The initial state, triangular system, W/U, pseudo-values and exit memory must be identical for two calls differing only in query scale.

Final demonstration to include with your submission

Test $T = 129$, $D_k = 128$, $D_v = 64$ with nonzero initial memory and unequal channel gates. Split at token 65 and carry state between calls. Compare concatenated outputs and final memory with a full call; report dtype, toolchain, both errors, and reused primitives. Public tolerances remain provisional.

References

[1] Kimi Team. *Kimi Linear: An Expressive, Efficient Attention Architecture*. 2025. https://arxiv.org/abs/2510.26692